# nomic-ai/nomic-embed-text-v1

like 582 | Follow Nomic AI 1.22k

Sentence Similarity | sentence-transformers | PyTorch | ONNX | Safetensors | Transformers | Transformers.js | English | nomic_bert | feature-extraction | mteb | custom_code | Eval Results (legacy) | text-embeddings-inference | arxiv:2402.01613 | License: apache-2.0

Deploy | Copy to bucket NEW | Use this model

Model card | Files xet | Community 31

A newer version of this model is available: nomic-ai/nomic-embed-text-v1.5

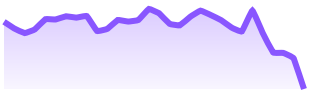
## nomic-embed-text-v1: A Reproducible Long Context (8192) Text Embedder

Blog | Technical Report | AWS SageMaker | Atlas Embedding and Unstructured Data Analytics Platform

`nomic-embed-text-v1` is 8192 context length text encoder that surpasses OpenAI text-embedding-ada-002 and text-embedding-3-small performance on short and long context tasks.

## Performance Benchmarks

Downloads last month
**3,945,262**



### Safetensors

Model size | 0.1B params
Tensor type | F32 | Files info

### Inference Providers NEW

Sentence Similarity

This model isn't deployed by any Inference Provider.

8 Ask for provider support

### Model tree for nomic-ai/nomic-em...

Finetunes 22 models
Quantizations 6 models

### Spaces using nomic-ai/nomic... 100

mteb/leaderboard

| Name | SeqLen | MTEB | LoCo | Jina Long Context | Op We |
|---|---|---|---|---|---|
| nomic-embed-text-v1 | 8192 | **62.39** | **85.53** | 54.16 | ✅ |
| jina-embeddings-v2-base-en | 8192 | 60.39 | 85.45 | 51.90 | ✅ |
| text-embedding-3-small | 8191 | 62.26 | 82.40 | **58.20** | ❌ |
| text-embedding-ada-002 | 8191 | 60.99 | 52.7 | 55.25 | ❌ |

**Exciting Update!**: `nomic-embed-text-v1` is now multimodal! nomic-embed-vision-v1 is aligned to the embedding space of `nomic-embed-text-v1`, meaning any text embedding is multimodal!

## Usage

**Important**: the text prompt *must* include a *task instruction prefix*, instructing the model which task is being performed.

For example, if you are implementing a RAG application, you embed your documents as `search_document: <text here>` and embed your user queries as `search_query: <text here>`.

**Notice**: From transformers v5.5.0 and sentence transformers v5.3.0, `trust_remote_code=True` will

eduagarcia/multilingual-tokenizer-le…
mteb/leaderboard_legacy
LAWGPT/attorneygpt
amryassin/embedding-bench
MJ-Prod/Fiscal
sq66/leaderboard_legacy
reader-1/1
+ 92 Spaces

**Collection including** `nomic-ai/nom…`

**Nomic Embed** Collection
Open Sour… • 8 items • Upd… • △ 24

**Paper for** `nomic-ai/nomic-embed-…`

Nomic Embed: Training a Reproduc…
Paper • 2402.01613 • Publi… • △ 18

**Evaluation results**

| | | | |
|---|---|---|---|
| accuracy … | test set | self-reported | 76.851 |
| ap on MTE… | test set | self-reported | 40.592 |
| f1 on MTE… | test set | self-reported | 71.016 |
| accuracy … | test set | self-reported | 91.519 |
| ap on MTE… | test set | self-reported | 88.503 |
| f1 on MTE… | test set | self-reported | 91.503 |
| accuracy … | test set | self-reported | 47.364 |
| f1 on MTE… | test set | self-reported | 46.727 |

Expand 982 evaluations

no longer be necessary. This will only be possible with the text-only series as of now.

### Task instruction prefixes

`search_document`

#### Purpose: embed texts as documents from a dataset

This prefix is used for embedding texts as documents, for example as documents for a RAG index.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['search_document: TSNE is a dime
embeddings = model.encode(sentences)
print(embeddings)
```

`search_query`

#### Purpose: embed texts as questions to answer

This prefix is used for embedding texts as questions that documents from a dataset could resolve, for example as queries to be answered by a RAG application.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['search_query: Who is Laurens va
embeddings = model.encode(sentences)
print(embeddings)
```

`clustering`

**Purpose: embed texts to group them into clusters**

This prefix is used for embedding texts in order to group them into clusters, discover common topics, or remove semantic duplicates.

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['clustering: the quick brown fo
embeddings = model.encode(sentences)
print(embeddings)
```

`classification`

**Purpose: embed texts to classify them**

This prefix is used for embedding texts into vectors that will be used as features for a classification model

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['classification: the quick brow
embeddings = model.encode(sentences)
print(embeddings)
```

### Sentence Transformers

```
from sentence_transformers import SentenceTra

model = SentenceTransformer("nomic-ai/nomic-
sentences = ['search_query: What is TSNE?',
embeddings = model.encode(sentences)
print(embeddings)
```

### Transformers

```
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, Autol

def mean_pooling(model_output, attention_mask
    token_embeddings = model_output[0]
    input_mask_expanded = attention_mask.unsq
    return torch.sum(token_embeddings * input

sentences = ['search_query: What is TSNE?',

tokenizer = AutoTokenizer.from_pretrained('bo
model = AutoModel.from_pretrained('nomic-ai/n
model.eval()

encoded_input = tokenizer(sentences, padding=

with torch.no_grad():
    model_output = model(**encoded_input)

embeddings = mean_pooling(model_output, encod
embeddings = F.normalize(embeddings, p=2, dim
print(embeddings)
```

The model natively supports scaling of the sequence length past 2048 tokens. To do so,

```
- tokenizer = AutoTokenizer.from_pretrained(
+ tokenizer = AutoTokenizer.from_pretrained(


- model = AutoModel.from_pretrained('nomic-ai
+ rope_parameters = {"rope_theta": 1000.0, "
+ model = AutoModel.from_pretrained('nomic-ai
```

### Transformers.js

```
import { pipeline } from '@xenova/transforme

// Create a feature extraction pipeline
const extractor = await pipeline('feature-ex
    quantized: false, // Comment out this li
});

// Compute sentence embeddings
const texts = ['search_query: What is TSNE?'
const embeddings = await extractor(texts, {
console.log(embeddings);
```

### Nomic API

The easiest way to get started with Nomic Embed is through the Nomic Embedding API.

Generating embeddings with the `nomic` Python client is as easy as

```
from nomic import embed

output = embed.text(
    texts=['Nomic Embedding API', '#keepAIOp
    model='nomic-embed-text-v1',
    task_type='search_document'
)

print(output)
```

For more information, see the API reference

### Training

Click the Nomic Atlas map below to visualize a 5M sample of our contrastive pretraining data!

We train our embedder using a multi-stage training pipeline. Starting from a long-context BERT model, the first unsupervised contrastive stage trains on a dataset generated from weakly related text pairs, such as question-answer pairs from forums like StackExchange and Quora, title-body pairs from Amazon reviews, and summarizations from news articles.

In the second finetuning stage, higher quality labeled datasets such as search queries and answers from web searches are leveraged. Data curation and hard-example mining is crucial in this stage.

For more details, see the Nomic Embed Technical Report and corresponding blog post.

Training data to train the models is released in its entirety. For more details, see the `contrastors` repository

## Join the Nomic Community

- Nomic: https://nomic.ai

- Discord: https://discord.gg/myY5YDR8z8
- Twitter: https://twitter.com/nomic_ai

## Citation

If you find the model, dataset, or training code useful, please cite our work

```
@misc{nussbaum2024nomic,
      title={Nomic Embed: Training a Reprodu
      author={Zach Nussbaum and John X. Morr
      year={2024},
      eprint={2402.01613},
      archivePrefix={arXiv},
      primaryClass={cs.CL}
}
```